CSE 3100

C Programming Practice Problems & Worked Solutions

16 problems • Answers • Step-by-step reasoning • Concept quiz

Based on: sum.c, Leibniz.c, taxicab-number.c, dice.c, hex.c, sum_n_bytes.c, shorest.c, and swap.c.

Study tip: Cover the green answer paragraph while solving each question. Then check the explanation.

Loops and arithmetic

1. Predict the output

What prints?

```
int i = 1, sum = 0;
while (i <= 5) {
    sum += i;
    i++;
}
printf("%d\n", sum);
```

Answer: 15

How to solve: The loop adds 1, 2, 3, 4, and 5. The running totals are 1, 3, 6, 10, 15. When i becomes 6, the condition fails.

2. Integer overflow

Why is this incorrect for a typical 32-bit int? Fix it.

```
int n = 100000, sum = 0;
for (int i = 1; i <= n; i++) sum += i;
printf("%d\n", sum);
```

Answer: The mathematical sum is 5,000,050,000; signed int overflow causes undefined behavior.

How to solve: Using n(n+1)/2 gives 100000 * 100001 / 2 = 5,000,050,000. This exceeds 2,147,483,647. Declare long long sum = 0; and print it with %lld.

3. Debug the pi series

What is wrong with this series calculation?

```
double sum = 0;
for (int i = 0; i < 10000; i++)
    sum += 8 / ((4*i+1)*(4*i+3));
```

Answer: The expression uses integer division, truncating fractional terms.

How to solve: At i=0, integer 8/3 evaluates to 2 rather than 2.666...; for i=1, integer 8/35 evaluates to 0. Use 8.0 / ((4.0*i+1.0)*(4.0*i+3.0)).

4. Count nested-loop operations

How many lines print?

```
for (int i = 1; i <= 3; i++)
    for (int j = i; j <= 3; j++)
        printf("%d %d\n", i, j);
```

Answer: 6 lines

How to solve: i=1 gives three pairs (1,1),(1,2),(1,3); i=2 gives two; i=3 gives one. Total 3+2+1=6.

Arrays and memory

5. Convert decimal to hexadecimal

Convert 254 to hexadecimal. What is stored before reversal?

Answer: FE; before reversal, the characters are E then F.

How to solve: 254 % 16 = 14 (E), then 254 / 16 = 15; 15 % 16 = 15 (F), then quotient 0. Reverse EF to get FE.

6. Complete dec_hex

Fill the missing logic for nonnegative d, given char digits[] = "0123456789ABCDEF".

```
int k = 0;
if (d == 0) hex[k++] = '0';
while (d > 0) {
    // TODO: append digit and update d
}
// TODO: reverse and terminate
```

Answer: Use remainder, repeated division, reversal, and a null terminator.

How to solve: Append hex[k++] = digits[d % 16]; then d /= 16. Swap hex[i] and hex[k-1-i] for i < k/2. Finally set hex[k] = '\0'.

7. Sum the first four bytes

For 4-byte unsigned integers on a little-endian machine, p[0]=1, p[1]=256. What does sum_n_bytes(p,4) return?

```
unsigned char *bytes = (unsigned char *)p;
unsigned sum = 0;
for (int i = 0; i < 4; i++) sum += bytes[i];
```

Answer: 1

How to solve: p[0]=1 is stored as bytes 01 00 00 00. The function sums the first 4 bytes, not 4 integers, so 1+0+0+0=1. For n=8, the sum would be 2.

8. Pointer arithmetic

Suppose p is int * at address 1000 and sizeof(int)=4. What are p+1 and (char *)p+1?

Answer: 1004 and 1001, respectively.

How to solve: Adding 1 to an int * advances one int (4 bytes), while adding 1 to a char * advances one byte.

Pointers and structures

9. Swap two pointers

What print values result after swap(&pa, &pb)?

```
void swap(int **p1, int **p2) {
    int *tmp = *p1;
    *p1 = *p2;
    *p2 = tmp;
}
int a=10, b=20;
int *pa=&a, *pb=&b;
swap(&pa, &pb);
```

Answer: *pa = 20, *pb = 10; a and b remain 10 and 20.

How to solve: The function gets addresses of the pointer variables and exchanges which ints they point at. It does not exchange the int values themselves.

10. Swap structures

What are x and y after calling swap_person(&x,&y)?

```
typedef struct { unsigned id; char name[64]; } person_t;
person_t x={1,"Alice"}, y={2,"Bob"};
void swap_person(person_t *a, person_t *b) {
    person_t tmp = *a;
    *a = *b;
    *b = tmp;
}
```

Answer: x = {2, "Bob"}; y = {1, "Alice"}.

How to solve: tmp holds a full copy of x. Assigning *a=*b copies the complete y structure to x; *b=tmp restores old x in y.

11. Find the closest pair

Which two of these points have the smallest Euclidean distance?

```
(id 0: 0,0)   (id 1: 3,4)   (id 2: 1,1)
```

Answer: Points 0 and 2; distance sqrt(2).

How to solve: Squared distances: d(0,1)^2=25, d(0,2)^2=2, d(1,2)^2=13. Minimum is 2; square root is unnecessary for choosing a pair.

12. Analyze time complexity

What is the runtime order?

```
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++)
        squared_dist(&points[i], &points[j]);
```

Answer: O(n^2)

How to solve: The inner loop runs n-1, n-2, ..., 1 times. The total is n(n-1)/2, quadratic in n.

Simulation and search

13. Three dice probability

Find the exact probability that three fair six-sided dice total at least 17.

Answer: 4/216 = 1/54 = approximately 0.01852 (1.85%).

How to solve: There are 6^3=216 ordered outcomes. Sum 18: (6,6,6), one. Sum 17: permutations of (5,6,6), three. Thus four successful outcomes.

14. Floating-point probability

Why cast count to double?

```
long count = 25, trials = 100;
double result = (double)count / trials;
```

Answer: It produces 0.25 rather than integer quotient 0.

How to solve: Without the cast, 25/100 truncates to 0 before assignment. Cast before division, not around the result.

15. Taxicab number

Why start the inner loop with j=i? What is the smallest number expressible as two positive cubes in two distinct unordered ways?

Answer: To avoid counting symmetric duplicates; the answer is 1729.

How to solve: 1729=1^3+12^3=9^3+10^3. Enforcing i<=j prevents (1,12) and (12,1) from being counted as different ways.

Pointers and structures

16. Fix pass-by-value swapping

Why does this not swap caller variables, and how do you fix it?

```
void swap(int a, int b) {
    int tmp = a; a = b; b = tmp;
}
int x=5, y=9;
swap(x,y);
```

Answer: It only swaps local copies; x=5 and y=9 afterward.

How to solve: C passes arguments by value. Change parameters to int *a, int *b; swap *a and *b; then call swap(&x,&y). This changes x to 9 and y to 5.

Quick concept quiz

1. What is = vs ==?

Answer: = assigns; == compares for equality.

2. What does &x mean?

Answer: The address of x.

*3. What does *p mean?*

Answer: Access the value at the address stored in p.

4. malloc vs calloc?

Answer: malloc allocates uninitialized memory; calloc zero-initializes its allocated bytes.

5. Why free()?

Answer: To release dynamically allocated storage.

6. Purpose of \0?

Answer: Null terminator marking the end of a C string.

7. What does srand(12345) do?

Answer: Sets a fixed pseudorandom seed for reproducible sequences on a given implementation.

8. What does break do?

Answer: Exits the innermost loop or switch.

Exam review priorities

1. Pointers: addresses, dereferencing, pointer-to-pointer, and swapping.
2. Loops: tracing output, nested-loop counts, and early exits.
3. Arrays and bytes: indexing, hexadecimal conversion, and null terminators.
4. Structures: field access with . versus ->, copying, and passing pointers.
5. Numeric types: integer division, overflow, and format specifiers.
6. Algorithm analysis: squared distance and O(n^2) nested loops.

Remember: p->x is equivalent to (*p).x when p points to a structure.